

Python pour le trading — Document 3/8

Les concepts intermédiaires distinctifs

1. Les decorators

Un **decorator** (décorateur) est une fonction qui en **enveloppe** une autre pour lui ajouter un comportement, sans modifier son code. On l'a déjà croisé sans le nommer : `@property` et `@dataclass` au document 2 étaient des décorateurs. Le principe : on « emballe » une fonction dans une autre qui s'exécute autour d'elle.

1.1 L'idée de base

Imaginons qu'on veuille mesurer le temps d'exécution de plusieurs fonctions de trading, sans réécrire chacune. Un décorateur le fait une fois pour toutes.

```
import time

def chronometre(fonction):
    def enveloppe(*args, **kwargs):
        debut = time.time()
        resultat = fonction(*args, **kwargs)  # on appelle la fonction d'origine
        duree = time.time() - debut
        print(f"{fonction.__name__} a pris {duree:.4f}s")
        return resultat
    return enveloppe

@chronometre
def calculer_signaux(prix):
    return sum(prix) / len(prix)

moyenne = calculer_signaux([5230, 5231, 5232])
# affiche : calculer_signaux a pris 0.0000s
print(moyenne)
```

Décortiquons. `chronometre` reçoit une fonction en argument et renvoie une nouvelle fonction, `enveloppe`, qui exécute du code **avant** et **après** l'appel d'origine (ici, mesurer le temps). La ligne `@chronometre` au-dessus de `calculer_signaux` est un raccourci : elle signifie « remplace `calculer_signaux` par sa version enveloppée ». On retrouve `*args`, `**kwargs` du document 1 : ils permettent à l'enveloppe d'accepter n'importe quels arguments et de les transmettre tels quels à la fonction d'origine.

Les deux return, dans l'ordre d'exécution

La présence de **deux** `return` déroute souvent. La clé est qu'ils ne s'exécutent **pas** au même moment ni le même nombre de fois. Voici la chronologie complète :

1. Python lit la ligne `@chronometre` placée au-dessus de la fonction.
2. Il appelle `chronometre(calculer_signaux)` **une seule fois**, au moment de la définition.

3. `return enveloppe` **(B)** s'exécute alors une fois : il remplace `calculer_signaux` par `enveloppe`.
1. Plus tard, chaque appel à `calculer_signaux(...)` exécute en réalité `enveloppe(...)`.
2. Dans l'`enveloppe`, `resultat` reçoit la valeur renvoyée par la vraie fonction.
3. `return resultat` **(A)** s'exécute à **chaque** appel et transmet ce résultat à l'appelant — c'est pourquoi `moyenne` vaut bien `5231.0`.

En résumé : `return enveloppe` sert une fois à installer le décorateur ; `return resultat` sert à chaque appel à restituer la valeur calculée. Aucun des deux n'est « perdu » : ils opèrent simplement à deux moments différents.

1.2 Un décorateur utile : le retry

En trading, une connexion à une API peut échouer ponctuellement. Un décorateur peut réessayer automatiquement, sans alourdir chaque fonction. L'exemple ci-dessous est complet : on simule des échecs, puis on appelle réellement la fonction décorée pour voir le comportement.

```
import random

def retry(n_essais):
    # décorateur paramétré
    def decorateur(fonction):
        def enveloppe(*args, **kwargs):
            for tentative in range(n_essais):
                try:
                    return fonction(*args, **kwargs)  # succès : on sort tout de
suite
                except ConnectionError:
                    print(f"Échec {tentative + 1}/{n_essais}, on réessaie...")
                    # si on arrive ici, les n_essais ont tous échoué
                    raise ConnectionError("Toutes les tentatives ont échoué")
            return enveloppe
        return decorateur

@retry(n_essais=3)
def recuperer_prix(symbole):
    # simulation : 70 % de chance d'échouer à chaque appel
    if random.random() < 0.7:
        raise ConnectionError("réseau indisponible")
    return 5231.0

# Appel réel : on voit les échecs successifs, puis le succès OU l'échec final
try:
    prix = recuperer_prix("ES")
    print(f"Prix obtenu : {prix}")
except ConnectionError as e:
    print(f"Abandon : {e}")

# Exemple de sortie possible :
# Échec 1/3, on réessaie...
# Échec 2/3, on réessaie...
```

```
# Prix obtenu : 5231.0
```

Ce décorateur prend lui-même un argument (`n_essais`), d'où le niveau d'imbrication supplémentaire : `reessayer` renvoie décorateur, qui renvoie enveloppe. À l'usage, `@reessayer(n_essais=3)` entoure `recuperer_prix` d'une boucle qui retente jusqu'à trois fois en cas de `ConnectionError`. Deux issues sont possibles : soit un appel réussit et `return fonction(...)` sort immédiatement de la boucle en renvoyant le prix ; soit les trois tentatives échouent, la boucle se termine, et la ligne `raise ConnectionError(...)` lève une erreur finale que l'appelant peut rattraper. La fonction métier, elle, reste propre : elle ne contient aucune logique de retry.

Pourquoi ça compte. Les décorateurs séparent la logique transversale (*timing, retry, journalisation, contrôle d'accès*) de la logique métier. Dans un système de trading, on décore les fonctions critiques pour les rendre robustes sans polluer leur code, ce qui est plus sûr et plus maintenable.

1.3 Préserver l'identité de la fonction : `functools.wraps`

Avant de voir le problème, il faut savoir que toute fonction Python porte deux étiquettes automatiques. `__name__` contient le **nom** de la fonction sous forme de texte. `__doc__` contient sa **docstring** : la chaîne de documentation placée en première ligne du corps de la fonction, entre triple guillemets, qui décrit ce qu'elle fait. Voyons-les sur une fonction **non** décorée :

```
def calculer_signaux(prix):
    """Calcule la moyenne d'une liste de prix.""" # <- ceci est la docstring
    return sum(prix) / len(prix)

print(calculer_signaux.__name__)
# affiche : calculer_signaux

print(calculer_signaux.__doc__)
# affiche : Calcule la moyenne d'une liste de prix.
```

Donc `__name__` donne `calculer_signaux` (le nom), et `__doc__` donne le texte `Calcule la moyenne d'une liste de prix.` (la docstring). Ces deux étiquettes servent au débogage, à l'aide intégrée (`help(calculer_signaux)`) et aux outils qui inspectent le code.

Le problème surgit dès qu'on décore la fonction. Comme vu en 1.1, `@chronometre` remplace `calculer_signaux` par la fonction interne `enveloppe`. Les étiquettes lues sont alors celles de `enveloppe`, pas celles de la fonction d'origine :

```
@chronometre
def calculer_signaux(prix):
    """Calcule la moyenne d'une liste de prix."""
    return sum(prix) / len(prix)

print(calculer_signaux.__name__)
# affiche : enveloppe (et non "calculer_signaux" !)

print(calculer_signaux.__doc__)
# affiche : None (la docstring d'origine est perdue)
```

On a donc perdu le vrai nom et la docstring : `__name__` renvoie maintenant enveloppe et `__doc__` renvoie None. C'est gênant pour le débogage et l'inspection, car la fonction « ment » sur son identité. La solution standard est `functools.wraps`, un décorateur qu'on applique à l'enveloppe pour qu'elle recopie les étiquettes de la fonction d'origine :

```
from functools import wraps

def chronometre(fonction):
    @wraps(fonction)                # recopie __name__, __doc__... de "fonction"
    def enveloppe(*args, **kwargs):
        return fonction(*args, **kwargs)
    return enveloppe

@chronometre
def calculer_signaux(prix):
    """Calcule la moyenne d'une liste de prix."""
    return sum(prix) / len(prix)

print(calculer_signaux.__name__)
# affiche : calculer_signaux          (le vrai nom est préservé)

print(calculer_signaux.__doc__)
# affiche : Calcule la moyenne d'une liste de prix.  (docstring préservée)
```

Grâce à `@wraps(fonction)`, l'enveloppe affiche désormais `calculer_signaux` et la bonne docstring, comme si elle n'avait jamais été décorée. Dans du vrai code Python, `@wraps(fonction)` est quasiment systématique sur tout décorateur sérieux : c'est un réflexe à prendre.

1.4 Deux autres usages : journalisation et contrôle d'accès

Le même patron — envelopper une fonction sans la modifier — répond à deux autres besoins transversaux classiques : **journaliser** chaque appel, et **vérifier une autorisation** avant d'exécuter.

La journalisation. Le décorateur enregistre l'appel et le résultat, autour de la fonction d'origine. `*args`, `**kwargs` lui permettent d'envelopper n'importe quelle signature.

```
import functools, logging
log = logging.getLogger("trading")

def journaliser(fonction):
    @functools.wraps(fonction)
    def enveloppe(*args, **kwargs):
        log.info("Appel de %s args=%s", fonction.__name__, args)
        resultat = fonction(*args, **kwargs)
        log.info("%s a renvoyé %s", fonction.__name__, resultat)
        return resultat
    return enveloppe
```

```
@journaliser
def passer_ordre(symbole, quantite):
    return {"statut": "execute", "symbole": symbole}
```

À chaque appel de `passer_ordre`, deux lignes de journal encadrent l'exécution, sans qu'une seule ligne de logging n'apparaisse dans la fonction métier. `@functools.wraps` préserve son nom (section 1.3).

Le contrôle d'accès. Comme le `retry` (1.2), c'est un décorateur **paramétré** (deux niveaux d'imbrication) : il vérifie un rôle avant d'autoriser l'appel.

```
def exige_role(role_requis):
    # décorateur paramétré (comme le retry)
    def decorateur(fonction):
        @functools.wraps(fonction)
        def enveloppe(utilisateur, *args, **kwargs):
            if utilisateur.get("role") != role_requis:
                raise PermissionError(f"Rôle « {role_requis} » requis")
            return fonction(utilisateur, *args, **kwargs)
        return enveloppe
    return decorateur
```

```
@exige_role("trader")
def annuler_tous_les_ordres(utilisateur):
    ...
    # atteint seulement si le rôle convient
```

Avant chaque appel, l'enveloppe compare le rôle de l'utilisateur à celui requis ; si l'autorisation manque, elle lève `PermissionError` et la fonction n'est jamais exécutée. La logique de sécurité reste hors de la fonction métier.

Quatre besoins transversaux — **timing, retry, journalisation, contrôle d'accès** — un seul patron : envelopper la fonction sans la modifier. C'est toute la force des décorateurs.

2. Les context managers (with)

Un **context manager** garantit qu'une **ressource** est correctement **ouverte puis refermée**, même en cas d'erreur. Par « ressource », on entend tout ce qui doit être libéré après usage : un **fichier** (à refermer), une **connexion réseau** à un courtier ou une base de données (à clôturer), un **verrou** (à relâcher), un **socket** ou un **fichier de log**. On le reconnaît au mot-clé `with`. C'est l'équivalent automatique du `try / finally` vu au document 1 : le nettoyage est garanti, on ne peut pas l'oublier.

2.1 L'usage le plus courant

```
# Sans context manager : on doit penser à fermer, même si une erreur survient
fichier = open("ordres.csv")
try:
    donnees = fichier.read()
finally:
    fichier.close()
    # facile à oublier
```

```
# Avec context manager : la fermeture est automatique
with open("ordres.csv") as fichier:
    donnees = fichier.read()
# ici, le fichier est déjà refermé, quoi qu'il arrive
```

Le bloc `with open(...) as fichier` ouvre le fichier, le rend disponible sous le nom `fichier`, et le referme automatiquement à la sortie du bloc — que tout se passe bien ou qu'une erreur survienne. On ne peut plus oublier le `close()`. C'est plus court et plus sûr que la version `try / finally`.

2.2 Écrire son propre context manager

On peut créer un context manager pour nos propres ressources. Exemple : une connexion à un courtier qu'on veut toujours fermer proprement.

```
from contextlib import contextmanager

@contextmanager
def connexion_courtier(nom):
    print(f"Connexion à {nom}...")
    connexion = ouvrir_connexion(nom)      # mise en place (à l'entrée du with)
    try:
        yield connexion                    # on "prête" la ressource au bloc with
    finally:
        connexion.fermer()                # nettoyage garanti (à la sortie du
with)
    print("Connexion fermée")

with connexion_courtier("IBKR") as conn:
    conn.envoyer_ordre(...)               # on utilise la connexion
# ici, la connexion est fermée automatiquement
```

Pourquoi `yield` et pas `return` ?

La différence est essentielle. Un `return` terminerait la fonction définitivement : il n'y aurait alors plus aucun moyen d'exécuter du code **après** que le bloc `with` a fini. Le `yield`, lui, **met la fonction en pause** à cet endroit précis et rend la main au bloc `with`. Cela découpe la fonction en deux moitiés exécutées à deux moments distincts :

- tout ce qui précède le `yield` s'exécute **à l'entrée** du bloc (l'ouverture) ;
- la valeur cédée par `yield connexion` est ce qu'on récupère après le `as` ;
- tout ce qui suit le `yield` s'exécute **à la sortie** du bloc (la fermeture).

Quand le `finally` s'exécute-t-il, et en cas d'erreur ?

Le `finally` s'exécute **dans tous les cas** à la sortie du bloc `with`. Concrètement :

- si le bloc se termine normalement, Python reprend la fonction juste après le `yield` et exécute le `finally` ;
- si une **erreur** survient à l'intérieur du bloc `with` (par exemple `conn.envoyer_ordre(...)` échoue), l'exception est « réinjectée » au niveau du `yield`. Le `finally` s'exécute alors

quand même — la connexion est fermée — **puis** l'exception continue de se propager normalement vers l'appelant.

C'est exactement le comportement du `try / finally` : le nettoyage est garanti que tout se passe bien ou mal. Le décorateur `@contextmanager` transforme simplement cette fonction à un `yield` en un objet utilisable avec `with`. On reconnaît `yield`, qu'on détaille à la section suivante.

Pourquoi ça compte. Une connexion à un courtier, un verrou, un fichier de log : autant de ressources qui **DOIVENT** être libérées. Le context manager rend cette libération automatique et infaillible, évitant les fuites de ressources qui, dans un système tournant en continu, finiraient par tout bloquer.

3. Les générateurs (yield)

Un **générateur** est une fonction qui produit ses valeurs **une par une**, à la demande, au lieu de tout calculer et tout renvoyer d'un coup. Le mot-clé qui le caractérise est `yield` (« céder »). On a effleuré l'idée au document 1 avec l'expression génératrice ; ici, c'est la version fonction.

D'abord, comment une boucle `for` consomme un générateur

Point crucial pour lever toute confusion : quand on écrit `for prix in flux_de_prix(...)`, il n'y a **pas** un seul appel. La boucle `for` appelle **automatiquement et en coulisse** la fonction `next()` une fois par tour — donc potentiellement des centaines de milliers de fois. Chaque `next()` relance le générateur, qui s'exécute jusqu'au `yield` suivant, cède une valeur, **puis se met en pause**. Le mécanisme exact (`iter()` / `next()` / `StopIteration`) est détaillé à la section 4 ; on en a juste besoin ici pour lire correctement l'exemple.

3.1 `return` contre `yield`

Pour bien voir la différence, on utilise volontairement un petit `range(3)` et on **affiche** chaque valeur. Repère bien qu'il y a **deux boucles** `for` distinctes : une **à l'intérieur** de chaque fonction, et une **à l'extérieur** qui consomme le résultat.

```
# ---- Version return : calcule TOUTE la liste d'un coup, la garde en mémoire ----
def tous_les_prix(n):
    resultat = []
    for i in range(n):          # BOUCLE INTERNE A : remplit la liste
        resultat.append(5230 + i)
    return resultat            # renvoie la liste complète, une seule fois

# BOUCLE EXTERNE : parcourt la liste DÉJÀ construite et stockée en mémoire
for prix in tous_les_prix(3):
    print(prix)

# Affiche :
# 5230
# 5231
# 5232

# ---- Version yield : produit chaque prix à la demande, sans rien stocker ----
def flux_de_prix(n):
    for i in range(n):          # BOUCLE INTERNE B : du générateur
```

```

        yield 5230 + i                # cède UNE valeur, puis se met en pause ICI

# BOUCLE EXTERNE : déclenche le générateur valeur par valeur
for prix in flux_de_prix(3):
    print(prix)

# Affiche EXACTEMENT la même chose :
# 5230
# 5231
# 5232

```

Les deux versions **affichent exactement la même chose** — 5230, 5231, 5232, un par ligne à cause des `print(prix)`. Ce qui change est invisible à l'écran mais capital en mémoire : `tous_les_prix(3)` construit d'abord la liste complète `[5230, 5231, 5232]`, la stocke, puis la **boucle externe** la parcourt. `flux_de_prix(3)`, lui, ne stocke jamais de liste : il fabrique chaque valeur juste au moment où la boucle externe la réclame.

Comment les deux boucles `for` s'enclenchent l'une l'autre

Le point qui prête à confusion est le mot « pause ». Suivons pas à pas ce qui se passe avec `for prix in flux_de_prix(3)` — la **boucle externe** :

1. La boucle externe démarre et réclame une première valeur (en coulisse, un `next()`). Cela lance `flux_de_prix`.
2. À l'intérieur, la **boucle interne B** `for i in range(n)` fait son premier tour (`i = 0`), atteint `yield 5230 + 0`, **cède** 5230 à la boucle externe, et **se met en pause sur cette ligne**.
3. La boucle externe reçoit 5230, exécute `print(prix)` → affiche 5230, puis réclame la valeur suivante (`next()` à nouveau).
4. Le générateur **reprend là où il s'était arrêté** — juste après le `yield` — et fait le tour suivant de la boucle interne (`i = 1`), cède 5231, se remet en pause.
5. La boucle externe affiche 5231, réclame encore : `i = 2`, cède 5232.
6. Au tour suivant, la boucle interne est épuisée (`range(3)` est fini) : le générateur se termine, ce qui signale la fin à la boucle externe, qui s'arrête.

Donc « elle reprend au prochain appel » ne désigne **pas** un nouvel appel à `flux_de_prix(3)` (celui-ci n'a lieu qu'une seule fois, au début). Le « prochain appel » est le `next()` implicite que la **boucle externe** envoie à chaque tour. La boucle interne (B, dans le générateur) et la boucle externe avancent **ensemble**, un cran chacune à la fois : la boucle interne produit une valeur, se fige, la boucle externe la consomme avec `print`, puis réveille la boucle interne pour la valeur d'après. C'est ce va-et-vient qui distingue `yield` d'un `return`, lequel exécuterait toute la boucle interne d'un trait avant de rendre quoi que ce soit.

3.2 Pourquoi c'est crucial pour les gros volumes

Imaginons qu'on lise des millions de ticks depuis un fichier. Tout charger en mémoire est impossible ; un générateur les traite à la volée.


```
def lire_ticks(chemin):
    # "chemin" = chemin vers le fichier
    with open(chemin) as fichier:
        for ligne in fichier:
            # le fichier se lit ligne par ligne
            symbole, prix = ligne.split(",")
            yield symbole, float(prix) # une donnée à la fois, jamais tout en
mémoire

# On passe le chemin du fichier en argument ; le nom du paramètre est "chemin",
# la valeur est ici le chemin "enormes_donnees.csv".
for symbole, prix in lire_ticks("enormes_donnees.csv"):
    analyser(symbole, prix)
```

Note sur le nommage : le paramètre s'appelle `chemin` parce qu'il reçoit un **chemin de fichier** (une chaîne de caractères comme `"enormes_donnees.csv"`), pas le fichier lui-même. C'est `open(chemin)` qui ouvre réellement le fichier situé à ce chemin. Le `.csv` n'est qu'une extension : `chemin` reste un nom générique correct, qui fonctionnerait avec n'importe quel format texte.

Ce générateur ne garde jamais qu'une ligne à la fois en mémoire, quel que soit le nombre total de lignes. Couplé au context manager `with` (section 2), il lit un fichier gigantesque proprement. C'est le motif standard pour traiter des données de marché qui dépassent la taille de la mémoire.

Pourquoi ça compte. Les données de marché se comptent en millions, voire milliards, de points. Les charger en entier saturerait la mémoire. Les générateurs permettent de les traiter en flux, un élément à la fois — c'est ce qui rend possible l'analyse de gros historiques ou de flux temps réel.

4. Les itérateurs et le protocole d'itération

Un **itérateur** est un objet qui sait fournir ses éléments un par un, ce qui le rend utilisable dans une boucle `for`. Comprendre ce mécanisme éclaire ce qui se passe « sous le capot » de chaque boucle Python — et les générateurs de la section 3 en sont justement un cas particulier.

```
# Ce que Python fait RÉELLEMENT quand on écrit : for x in collection
itérateur = iter(collection)      # 1. obtenir un itérateur
while True:
    try:
        x = next(itérateur)        # 2. demander l'élément suivant
    except StopIteration:          # 3. plus d'éléments -> on sort
        break
    traiter(x)                     # 4. utiliser l'élément
```

Une boucle `for` n'est donc qu'un raccourci : `iter()` obtient un itérateur à partir de la collection, puis `next()` en tire les éléments un à un, jusqu'à ce qu'une exception `StopIteration` signale la fin. C'est précisément ce `next()` implicite, répété à chaque tour, qu'évoquait la section 3. Un générateur est exactement cela : un itérateur dont `yield` produit les `next()` successifs et dont la fin lève automatiquement `StopIteration`. C'est pourquoi on peut écrire `for prix in flux_de_prix(...)` directement.

Le protocole sous-jacent : `__iter__` et `__next__`

Les fonctions intégrées `iter()` et `next()` ne font que déléguer à deux méthodes spéciales de l'objet : `iter(obj)` appelle en réalité `obj.__iter__()`, et `next(it)` appelle `it.__next__()`. Un

objet qui implémente ces deux méthodes est un itérateur ; c'est le pont direct vers la POO du prochain document, où l'on écrira nos propres objets itérables.

Itérateur vs iterable : une distinction d'entretien

Attention à ne pas confondre les deux. Un **iterable** est un objet sur lequel on peut itérer : il sait **fournir** un itérateur quand on lui demande (via `__iter__()`). Un **itérateur** est l'objet qui **produit effectivement** les valeurs une à une (via `__next__()`) et qui finit par lever `StopIteration`.

Concrètement, les collections de base — `list`, `tuple`, `dict`, `set` — sont des **iterables**, **pas** des itérateurs : on ne peut pas leur appliquer `next()` directement, mais `iter(ma_liste)` en fabrique un itérateur à la demande. Un générateur, lui, est **à la fois** un iterable et son propre itérateur. Cette distinction est très souvent demandée en entretien.

```
ma_liste = [10, 20, 30]
next(ma_liste)           # TypeError : une liste n'est PAS un itérateur

it = iter(ma_liste)      # iter() fabrique un itérateur à partir de l'iterable
print(next(it))          # -> 10
print(next(it))          # -> 20
```

Pourquoi ça compte. Comprendre le protocole d'itération démystifie les boucles et les générateurs. Cela permet de manipuler des flux de données paresseux (lazy) en toute confiance, et d'écrire des objets personnalisés qui s'utilisent naturellement dans un `for`.

5. Les closures et la portée des variables (LEGB)

La **portée** (scope) détermine où une variable est visible. Python suit la règle **LEGB**, qui dit dans quel ordre il cherche un nom. Et une **closure** est une fonction qui « se souvient » de variables de son environnement de création. Ces deux notions expliquent des comportements parfois déroutants, dont le piège vu au document 1.

5.1 La règle LEGB

Quand Python rencontre un nom de variable, il le cherche dans cet ordre :

```
L - Local      : à l'intérieur de la fonction courante
E - Enclosing  : dans la fonction englobante (si imbriquée)
G - Global     : au niveau du module (le fichier)
B - Built-in   : les noms intégrés de Python (print, len, sum...)
```

```
prix_base = 5000           # G : variable globale

def ajuster():
    prime = 200             # L : variable locale à ajuster
    return prix_base + prime # prix_base trouvé au niveau Global

ajuster()                  # -> 5200
```

Dans `ajuster`, le nom `prime` est trouvé en Local, et `prix_base`, absent localement, est cherché plus haut et trouvé en Global. Python remonte les niveaux `L` → `E` → `G` → `B` jusqu'à trouver le nom ; s'il ne le trouve nulle part, il lève une `NameError`.

5.2 Une closure

Une closure naît quand une fonction interne capture une variable de la fonction qui l'entoure. Exemple : fabriquer des validateurs de prix sur mesure.

```
def creer_plafond(limite):          # fonction englobante
    def verifier(prix):            # fonction interne
        return prix <= limite      # "limite" vient du niveau Enclosing
    return verifier                # on renvoie la fonction interne

plafond_es = creer_plafond(5300)   # une closure qui "retient" limite=5300
plafond_es(5250)                   # -> True
plafond_es(5400)                   # -> False
```

La fonction `verifier` utilise `limite`, qui appartient à la fonction englobante `creer_plafond` (niveau `Enclosing` dans `LEGB`). Même après que `creer_plafond` a terminé, `plafond_es` « se souvient » que `limite` valait 5300 : c'est la closure. Cela permet de fabriquer des fonctions spécialisées à la volée. C'est aussi le mécanisme qui sous-tend les décorateurs de la section 1.

5.3 Modifier une variable capturée : `nonlocal`

Une closure peut **lire** librement une variable du niveau englobant. Mais dès qu'elle tente de la **modifier** par affectation, Python la considère par défaut comme une **nouvelle** variable locale — ce qui produit une `UnboundLocalError`. Le mot-clé `nonlocal` lève l'ambiguïté : il dit explicitement « cette variable est celle de la fonction englobante, pas une nouvelle locale ».

```
def compteur():
    n = 0                                # variable du niveau Enclosing

    def incrementer():
        nonlocal n                      # "n" désigne le n ci-dessus, pas un nouveau
    local
        n += 1                          # sans nonlocal : UnboundLocalError
        return n

    return incrementer

suivant = compteur()
print(suivant())                        # -> 1
print(suivant())                        # -> 2
print(suivant())                        # -> 3
```

Ici, `incrementer` est une closure qui **conserve un état** entre les appels grâce à `n`. Sans `nonlocal`, la ligne `n += 1` serait interprétée comme la création d'un `n` local lu avant d'être défini, d'où l'erreur. `nonlocal` est ainsi le complément naturel des closures : il permet non seulement de se souvenir d'une variable, mais aussi de la **faire évoluer**. (À distinguer de `global`, qui vise le niveau module, et non la fonction englobante.)

Au passage, cela éclaire le piège de l'argument mutable par défaut vu au document 1 : c'est le moment de **création** qui fige une valeur. Comprendre la portée, c'est comprendre quand et où les variables sont fixées.

Pourquoi ça compte. Les closures et la portée expliquent le comportement réel du code — quelle variable est lue, quand elle est figée, comment on la fait évoluer avec `nonlocal`. C'est indispensable pour déboguer des effets surprenants et pour comprendre les décorateurs, omniprésents dans le code professionnel.

6. Le type hinting (annotations de types)

Le **type hinting** consiste à annoter le code pour indiquer le type attendu des variables, des arguments et des valeurs de retour. Ces annotations sont **indicatives** : Python ne les vérifie pas à l'exécution, mais elles documentent le code et permettent à des outils (comme `mypy`) de détecter des erreurs avant même de lancer le programme. On les a vues au document 2 dans les `dataclasses` (`symbole: str`).

6.1 Annoter une fonction

```
# Sans annotations : on devine les types
def valeur_notionnelle(quantite, prix):
    return quantite * prix

# Avec annotations : les types sont explicites
def valeur_notionnelle(quantite: int, prix: float) -> float:
    return quantite * prix
```

La syntaxe : après chaque paramètre, `: type` indique son type attendu ; après les parenthèses, `-> float` indique le type de la valeur renvoyée. Ici, `quantite` est un entier, `prix` un nombre à virgule, et la fonction est censée renvoyer un `float`.

Une subtilité utile à signaler : ces annotations restent **purement indicatives**. Python ne les impose pas, et `quantite * prix` suit les règles normales de Python. Ainsi `valeur_notionnelle(5, 10)` renvoie `50` (un `int`), car `int * int` donne un `int` ; tandis que `valeur_notionnelle(5, 10.0)` renvoie `50.0` (un `float`). L'annotation `-> float` exprime l'intention — « cette fonction est faite pour manipuler des montants » — mais ne convertit rien automatiquement. Le code fonctionne identiquement sans les annotations ; avec elles, l'intention est claire pour quiconque lit la fonction et pour les outils d'analyse.

6.2 Types plus riches

Pour les collections et les cas plus complexes, on précise le contenu :

```
from typing import Optional

# une liste de nombres à virgule
def moyenne(prix: list[float]) -> float:
    return sum(prix) / len(prix)

# un dictionnaire symbole -> prix
def dernier_prix(table: dict[str, float], symbole: str) -> float:
    return table[symbole]

# Optional : peut renvoyer un float OU rien (None)
def chercher_prix(symbole: str) -> Optional[float]:
```

```
... # renvoie un float si trouvé, None sinon
```

On annote le contenu des collections (`list[float]`, `dict[str, float]`) et, avec `Optional[float]`, on indique honnêtement qu'une fonction peut renvoyer un float **ou** None — typiquement une recherche qui échoue.

Pourquoi ça compte. Dans un système de trading, les annotations de types documentent les contrats des fonctions (un prix est un float, un symbole une chaîne) et permettent à des outils de détecter, avant l'exécution, des erreurs qui coûteraient cher en production.

Quiz de révision

Sur les décorateurs

Q1. Qu'est-ce qu'un décorateur ?

Réponse. Une fonction qui en enveloppe une autre pour lui ajouter un comportement (timing, retry, journalisation...) sans modifier le code de la fonction d'origine.

Q2. Dans le décorateur chronometre, pourquoi y a-t-il deux return, et quand s'exécutent-ils ?

Réponse. `return enveloppe` s'exécute une seule fois, au moment où le décorateur est appliqué : il remplace la fonction d'origine par l'enveloppe. `return resultat` s'exécute à chaque appel : il renvoie à l'appelant la valeur calculée par la fonction d'origine.

Q3. À quoi sert `functools.wraps` ?

Réponse. À recopier les métadonnées (`__name__`, `__doc__`...) de la fonction d'origine vers l'enveloppe, qui sinon les écraserait. C'est un réflexe standard sur tout décorateur.

Sur les context managers

Q4. Que garantit le mot-clé `with` ?

Réponse. Que la ressource ouverte (fichier, connexion, verrou...) sera refermée automatiquement à la sortie du bloc, même si une erreur survient.

Q5. À quelle structure du document 1 le context manager ressemble-t-il ?

Réponse. Au `try / finally` : le nettoyage est garanti dans tous les cas. Le `with` le rend automatique et impossible à oublier.

Q6. Dans un context manager écrit avec `@contextmanager`, pourquoi `yield` plutôt que `return`, et que se passe-t-il en cas d'erreur ?

Réponse. `yield` met la fonction en pause et rend la main au bloc `with`, ce qu'un `return` ne permettrait pas. Ce qui précède le `yield` s'exécute à l'entrée ; la valeur cédée est récupérée après le `as` ; ce qui suit (dans un `finally`) s'exécute à la sortie. En cas d'erreur dans le bloc, l'exception remonte au `yield`, le `finally` s'exécute quand même, puis l'exception continue de se propager.

Sur les générateurs et itérateurs

Q7. Quelle est la différence entre `return` et `yield` ?

Réponse. `return` termine la fonction et renvoie tout d'un coup. `yield` produit une valeur, met la fonction en pause, puis reprend où elle s'était arrêtée au `next()` suivant. Le générateur ne stocke pas tout en mémoire.

Q8. Dans `for prix in flux_de_prix(...)`, combien de fois la fonction est-elle relancée ?

Réponse. Autant de fois qu'il y a d'éléments : la boucle `for` envoie un `next()` implicite à chaque itération, et chaque `next()` relance le générateur du `yield` précédent jusqu'au `yield` suivant. Il n'y a pas un seul appel mais une reprise par valeur produite.

Q9. Pourquoi les générateurs sont-ils cruciaux pour les données de marché ?

Réponse. Parce que les données se comptent en millions de points. Un générateur les traite une par une sans tout charger en mémoire, ce qui permet d'analyser des fichiers ou flux trop gros pour la RAM.

Q10. Que fait Python en coulisse quand on écrit `for x in collection` ?

Réponse. Il appelle `iter()` pour obtenir un itérateur, puis `next()` pour tirer les éléments un par un, jusqu'à ce qu'une exception `StopIteration` signale la fin.

Q11. Quelle est la différence entre un iterable et un itérateur ?

Réponse. Un iterable sait fournir un itérateur (via `__iter__()`) : `list`, `tuple`, `dict`, `set` en sont. Un itérateur produit effectivement les valeurs une à une (via `__next__()`) et lève `StopIteration` à la fin. On ne peut pas faire `next()` sur une liste, mais `iter(liste)` en fabrique un itérateur.

Sur les closures et la portée

Q12. Que signifie l'acronyme LEGB ?

Réponse. L'ordre dans lequel Python cherche un nom de variable : Local, Enclosing (fonction englobante), Global (module), Built-in (noms intégrés).

Q13. Qu'est-ce qu'une closure ?

Réponse. Une fonction interne qui « se souvient » de variables de la fonction qui l'entoure, même après que celle-ci a terminé. Elle permet de fabriquer des fonctions spécialisées à la volée.

Q14. À quoi sert `nonlocal` dans une closure ?

Réponse. À indiquer qu'une variable affectée dans la fonction interne désigne celle de la fonction englobante, et non une nouvelle variable locale. Sans lui, `n += 1` sur une variable capturée lèverait une `UnboundLocalError`. Il permet de faire évoluer un état entre les appels.

Q15. Quel concept du document 3 repose sur les closures ?

Réponse. Les décorateurs : l'enveloppe capture la fonction d'origine de son environnement englobant, ce qui est exactement une closure.

Sur le type hinting

Q16. Python vérifie-t-il les annotations de types à l'exécution ?

Réponse. Non. Elles sont indicatives : elles documentent le code et permettent à des outils externes (comme `mypy`) de détecter des erreurs avant l'exécution, mais Python ne les impose pas au moment de tourner. Ainsi `valeur_notionnelle(5, 10)` renvoie un `int` malgré l'annotation `-> float`.

Q17. Que signifie l'annotation `Optional[float]` ?

Réponse. Que la valeur peut être un `float` OU `None`. C'est la façon honnête d'indiquer qu'une fonction peut ne rien renvoyer (par exemple une recherche qui échoue).

Q18. Comment annoter une fonction qui prend une liste de nombres et renvoie un nombre ?

Réponse. `def moyenne(prix: list[float]) -> float`. Le `:` type annoté le paramètre, le `->` type annoté la valeur de retour.

Document suivant (4/8) : la concurrence et l'asynchrone — `threading`, `multiprocessing`, le GIL, et `asyncio/await` pour traiter plusieurs flux de marché en parallèle.